AI 产品作品集

从想法到实践

自己把产品做出来，也自己验证它到底对不对。

关于我

本科人力资源管理，硕士管理科学与工程。在美团做过运营、在联想做过 AI 产品助理，并独立完成两个从 0 到 1 的 AI 产品。

我习惯自己动手把产品做出来——**因为每一个判断，我都想让它经得起验证。**

两个案例

01 AI + 心理

Glimmer 智能体

面向校园的心理陪伴 Agent：用 AI 接住学生的求助，再用人工兜底把高风险接稳。 →

02 AI + Chat BI

DataTalk

让不懂 SQL 的运营用一句话取数，重点解决「问出来的数，能不能信」。 →

邮箱 m18193310560@163.com GitHub github.com/SpringLibra 简历 PDF 下载

案例 01

Glimmer 智能体

角色 个人 0-1 项目　　时间 2026.03 – 2026.06　　领域 AI + 校园心理陪伴

它最难的不是"聊得像人"，而是——当一个人真的处在高风险里时，怎么保证它一定接得住。

01

问题

目标用户是那些**不敢求助、也不知道该找谁**的学生。这个场景有一条硬边界：AI 不能替代心理咨询师。而一旦漏掉一个真正处在高风险里的人，代价无法接受。

所以我把产品的核心，从"能不能陪聊"拉回到一个更硬的问题：**怎么把风险接住，并且说清楚 AI 和人工各自的边界在哪。**

02

关键判断

- **分层响应：**把对话分成 CHAT / CONSULT / RISK 三层，普通聊天不强行做心理测评——既控制成本，也避免对正常人过度干预。
- **三级风险判定：**词典硬信号 → 模型结构化评估 → 启发式兜底；并设一个独立的 Safety Agent，对高风险回复拥有**一票否决权**。
- **处置不是终点：**高风险不进"黑名单"，而是进一条责任链——风险报告 → 个案创建 → 辅导员预警 → 人工确认。

03

系统架构

我把系统拆成四层，每层只解决一个问题——这样任何一层出问题，都不会让整条对话垮掉。

- **编排层 · 多 Agent：**理解、风险评估、上下文、回复各由一个专业 Agent 负责，再由协调 Agent 收敛。我坚持让**风险判定独立于对话生成**——如果让模型"一边聊天一边自己判断安不安全"，它很容易为了把话说圆而放过风险。
- **安全层 · 三层兜底：**词典硬信号（命中即高危，不依赖模型）→ 模型结构化评估 → 启发式兜底（模型挂了也不崩）。Safety Agent 还会对**候选回复一票否决**：高风险回复缺安全指引，直接打回重写。
- **检索层 · 动态路由：**普通闲聊不查知识库（省成本、更快），只有求助与风险才走"向量 + BM25 + 重排"的混合检索；向量服务不可用时自动降级为纯关键词，**对话不中断**。
- **处置层 · 异步责任链：**高风险不进"黑名单"，而进一条后台责任链——写台账、建个案、推预警，幂等、可重试、有死信，且**不阻塞学生端的流式回复**。

MCP 工具	**6 个**——写台账、建个案、发预警、确认个案、追加备注、发预警邮件。我把"处置链"的每个动作**标准化成工具**，而不是写死在流程里：Agent 只决定"要不要处置"，工具负责"怎么处置"。
Skill	**7 个场景技能**——学业压力、焦虑安抚、睡眠作息、高危安全计划、转介资源、转介摘要、基础支持。按场景加载，给回复加领域约束；写成 SKILL.md，规则可版本化、可单独评审。
Harness	**两层**——一层跑业务（GlimmerAgentHarness），一层做验证（6 套回归：风险 / 路由 / 技能 / RAG / API / 工具队列）。验证层用 mock 模型 + 临时 SQLite + 内存记忆，**不依赖真实 MySQL / Redis / 模型**，随时全链路自检。产品"怎么跑"和"怎么证明它没坏"是两件事。

`Python` `FastAPI` `事件驱动多 Agent（黑板 + 认领式调度）` `Redis` `MySQL` `Chroma` `BM25 + BGE-M3` `MCP` `SSE` `Docker`

04

取舍

我想提高高风险召回，通常只有两条路：放宽判定阈值，或者改模型。我先试了改 Prompt。

结果召回从 **83.3% 抬到了 87.5%**，但误报率从 **2.1% 猛涨到 8.3%**——新增的误报几乎全是"玩梗"的正常学生。这让我判断：光靠 Prompt"宁可高估"会误伤玩笑，正解是把玩笑、黑话的降权落到规则层。

于是我放弃了这条捷径，保留原方案。宁可承认"一次没解决"，也不上线一个会吓到正常人的版本。

05

验证

我用一套 96 例的安全评测集来量化"接不接得住"——48 条高风险样本 + 48 条正常样本，覆盖隐喻、黑话、躯体化、玩笑、否认等最难的类型。

83.3%	2.1%	33.3%
高风险召回（真模型）	误报率	纯规则基线召回 · 模型层带来 50pt 增量

这组数字里，真正说明问题的是那个 **33.3%**：如果只靠关键词硬匹配，十个人里只能接住三个。是模型层的评估，把它抬到了 83%。

判定阈值 → 召回 / 误报 取舍

拖动滑块，看同一条评测集在不同判定线下会变成什么样。



生产安全线（risk == HIGH） · 上线采用的判定线：召回 83%、误报仅 2%。取舍从这里开始——宁可多拦一点，也不漏掉高风险的人。

以上全部取自项目内的真实评测报告（SafetyEval，N=96）。此外：回复质量 pass 98.3%（N=60）；judge 阴性对照坏回复 12/12 全部判 fail；标注一致性 kappa 0.886。

06

下一步

- **落地规则层降权：**把"玩梗 / 黑话 + 无明确危险信号"标记出来降权，再跑一轮 SafetyEval 对比，把误报打回去、同时保住召回。
- **评测集扩容：**安全集 96 → 120，回复集 60 → 100+，尽量掺入真实会话日志。
- **补真实标注一致性：**现有 kappa 0.886 是"人 vs LLM 代理"，要找真人做双标。

下一个案例 DataTalk · B 端问数 →

案例 02

DataTalk

角色 个人 0-1 项目　　时间 2026.06 – 2026.08　　领域 AI + 商业智能（Chat BI）

让不会写 SQL 的运营，用一句话拿到数。难的不是"能不能问"，是"问出来的数，能不能信"。

01

问题

增长、渠道、内容运营人员不会写 SQL——取数要么等一张固定报表，要么排队等技术团队写查询，门槛高、链路长。

但这个产品真正要解决的不是"能不能问数"，而是**问出来的数准不准**。一个查得很快、但结果错的问数产品，比没有更危险。

02

关键判断

- **先解决"模型看到什么"：**不把几百个字段一次性塞给模型，而是用字段级检索 + Schema 关系图，只给它当下需要的那张子图。
- **权限做在检索层：**按角色加载可用数据表，权限贯穿检索、建图、SQL、工具四个环节——不是只在下游拦一道。
- **承认歧义：**业务口径有歧义时，宁可让产品发起澄清，也不让模型猜一个看起来合理的答案。

03

系统架构

我把这条问数链路，用 LangGraph 编成了一张**可分支、可中断、可恢复**的图——而不是一问一答的直线。

- **工作流：**请求先进预处理判定意图，再分三条路——闲聊直接答、基于已有结果做分析、或去查数据库。查库这条会经过"检索 Schema → （必要时）向人澄清 → 生成并执行 SQL"。用图来编排的关键收益是：**澄清能中断流程、等用户回答后再恢复**，这条能力直线链条给不了。
- **给模型看什么：**不把几百个字段全塞进去，而是字段级检索 + Schema 关系图，只喂它当下需要的子图——这是挡幻觉的第一道闸。
- **权限：**每个请求带一个访问范围，**工具按角色注册**——没权限的用户根本看不到对应的数据库工具；SQL 执行时再二次校验（只读、单语句、禁 SELECT *）。
- **把规则沉淀成 Skill：**"怎么写对一条 SQL"（粒度、口径、时间规则、返回列顺序）写成一份 SKILL.md，而不是塞进系统提示词——规则可版本化、可独立迭代。
- **查询契约：**模型执行前必须先声明一份 query_contract（粒度 / 来源表 / 条件 / 指标 / 返回列），让"它到底想查什么"变得可核对。

MCP 工具

每个数据库一个只读查询工具（query_数据库名），外加**当前时间、解析相对日期、生成柱状图、生成饼图**四个辅助工具。关键在注册方式：工具**按角色注册**——没权限的库，模型连工具都看不到，而不是"看到了再被拒绝"。

Skill ×2

database_query（怎么把一句话写成一条对的只读 SQL）与 **data_qa**（基于已有结果做分析、出报告）。规则写进 SKILL.md——可版本化、可独立迭代，不污染主提示词。

`Python` `FastAPI` `LangGraph（可中断 / 可恢复）` `DuckDB` `MCP` `Vue` `字段级 RAG + Schema 图` `角色权限 ACL`

04

取舍

我的评测做了两个明确的取舍，而且我把它们写在了明面上：

- **评的是"检索对了之后"：**评测给定标准的检索词，所以我测的是"检索正确后，模型能不能把 SQL 写对"，不是端到端。先把能控的部分测准，端到端留到下一版。
- **澄清没能纳入评测：**澄清机制我设计了，但 430 条用例基本都给了明确查询，澄清**实际 0 次触发**。这是我这套评测的一个盲区，我没有藏它。

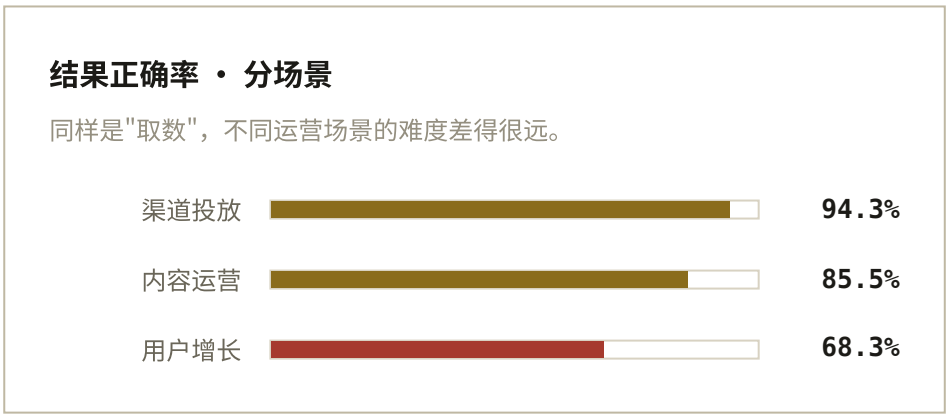
05

验证

我建了一套 430 条用例的评测集，覆盖增长、渠道、内容三类运营场景，从"字段召回 → SQL 执行 → 结果正确"一路量化到底。

95.26%	99.3%	82.56%
字段级 Schema 召回率	SQL 执行成功率	查询结果正确率

82.56% 不是一个漂亮的数字，但它有用——它让我把"取数不准"**定位到了具体场景和模块**，而不是笼统地说产品好不好。



再往下拆一层：简单查询正确率 86.6%，**复杂查询只有 33.3%**——问题集中爆发在**多表关联与聚合逻辑**上。这就是这套评测最值钱的产出：它没有给我一个高分，给我的是一个"该先修哪里"的答案。

数据取自项目内 evaluation/results/benchmark_live_all_curated（N=430）。

06

下一步

- **修多表关联与聚合：**复杂查询只有 33%，这是最该先动的地方。
- **把"澄清"纳入评测：**补一组"模糊意图"用例，让端到端链路真正被测到。
- **建口径层（指标字典）：**现在 Schema 精确率只有 19.44%——召回太宽。缺的正是一个统一的指标口径层，让"新增用户"这类词在不同角色下指向同一个定义。

看另一个案例 Glimmer · 风险分级 Agent →